SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Date:		**Duration:**	60 minutes

Code of Conduct

I P. MUNI KARTHIK / 1924250561 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollbac k details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/docum entation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

CI/CD PIPELINE DESIGN FOR DATA WAREHOUSE MANAGEMENT SYSTEM

1. Introduction

A Data Warehouse Management System (DWMS) is a software system used to collect, integrate, transform, store, and manage large volumes of data from multiple sources. It supports data analysis, reporting, business intelligence, and decision-making.

Since the system involves database operations, ETL processes, application code, configuration files, and data-quality rules, continuous integration and continuous deployment (CI/CD) can be used to automate software development, testing, quality verification, and deployment.

The proposed CI/CD pipeline automatically validates code changes, performs testing and quality checks, packages the application, and deploys it through development, staging, and production environments.

2. Project Requirements and CI/CD Needs

2.1 Functional Requirements

The Data Warehouse Management System should provide:

1. Data extraction from multiple sources.
2. Data transformation and cleaning.
3. Loading of processed data into the warehouse.
4. Database and schema management.
5. ETL/ELT workflow management.
6. Data validation and quality checking.
7. Data warehouse querying.
8. Report and dashboard generation.
9. User authentication and authorization.
10. Error logging and monitoring.
11. Backup and recovery.
12. Management of warehouse metadata.

2.2 Non-Functional Requirements

The system should provide:

- High reliability.
- Good performance.
- Scalability.
- Security.
- Data integrity.
- Maintainability.
- Availability.
- Fast deployment.
- Automated testing.
- Error detection.
- Easy rollback.

2.3 CI/CD Requirements

The CI/CD pipeline should:

- Automatically trigger when developers commit code.
- Maintain source code using Git.
- Build the application automatically.
- Validate database scripts.
- Execute automated tests.
- Perform code-quality analysis.
- Perform security scanning.
- Package the application.
- Deploy automatically to development.
- Deploy tested builds to staging.

3. Proposed CI/CD Pipeline Architecture

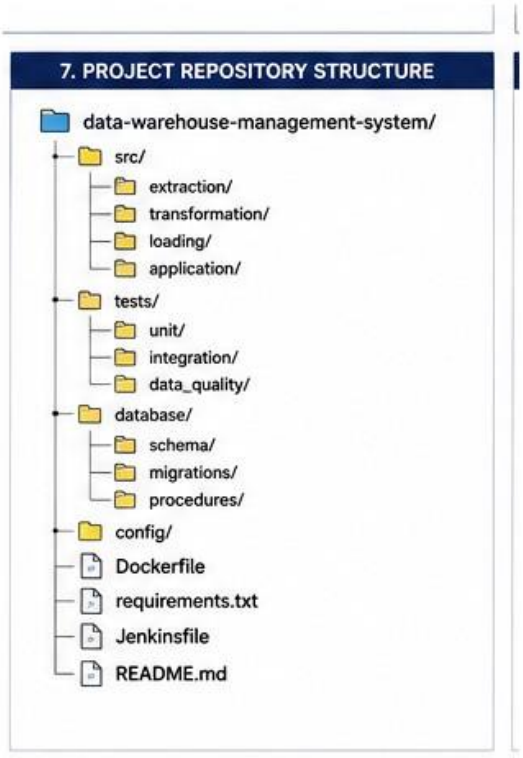
The proposed pipeline consists of the following stages:

4. Tools and Technologies

Pipeline StageTool / TechnologyPurpose		
Source Code Management	Git	Version control
Remote Repository	GitHub	Store source code
CI/CD Automation	Jenkins / GitHub Actions	Automate pipeline
Programming	Python / Java	Application development
Database	PostgreSQL / MySQL	Warehouse database
ETL	Python / Apache Airflow	Data processing
Build	Maven / Docker	Build and package application
Unit Testing	PyTest / JUnit	Test individual components
Integration Testing	PyTest / JUnit	Test system components together
Code Quality	SonarQube	Static code analysis
Security Scan	Trivy	Container vulnerability scanning
Containerization	Docker	Package application
Container Registry	Docker Hub / GitHub Container Registry	Store Docker images
Deployment	Docker / Kubernetes	Deploy application
Monitoring	Prometheus / Grafana	Monitor system
Pipeline StageTool / TechnologyPurpose		
Notification	Email / Slack	Pipeline notifications

---

5. Source Code Management



Git is used for version control and GitHub is used as the remote repository.

6. CI/CD Pipeline Stages

Stage 1 – Code Commit

The developer modifies the source code and commits the changes.

Example:

```
git add . git commit -m "Added customer
data transformation" git push
```

The push operation triggers the CI/CD pipeline.

Stage 2 – Source Code Validation

The pipeline checks:

- Repository structure.
- Required files.
- Syntax.
- Configuration.
- Dependency files.
- Database migration scripts.

Invalid code is rejected before moving to the next stage.

Stage 3 – Build

The application is built automatically.

Docker can be used to create a consistent application environment.

Example:

Dockerfile | v Docker Build | v Data Warehouse Application Image The Docker image contains:

- Application code.
- Required libraries.
- Runtime environment.
- Configuration required for execution.

---

7. Automated Testing Strategy

Testing is one of the most important parts of the CI/CD pipeline because errors in a Data Warehouse Management System can cause incorrect business reports and inaccurate analytical results.

7.1 Unit Testing

Unit testing checks individual functions.

Examples:

- Data extraction function.
- Data cleaning function.
- Data transformation function.
- Data validation function.
- Calculation functions.

Example:

Input:
Customer purchase = 1000

Expected transformed value:
1000

The test verifies that the transformation function produces the expected result.

7.2 Integration Testing

Integration testing verifies communication between different components.

Examples:

Data Source

↓

Extraction

↓

Transformation

↓

Warehouse Database

The test verifies that data successfully moves through the complete pipeline.

7.3 Database Testing

Database testing verifies:

- Tables.
- Primary keys.
- Foreign keys.
- Constraints.
- Stored procedures.
- Views.
- Database migrations.
- SQL queries.

It also verifies that database changes do not break existing functionality.

7.4 Data Quality Testing

Data quality testing is especially important for a Data Warehouse Management System.

The pipeline checks:

- Missing values.
- Duplicate records.
- Invalid values.
- Incorrect data types.
- Referential integrity.
- Record counts.
- Null values.
- Data consistency.

Example:

Source Records = 10,000
Loaded Records = 10,000

Result: PASS

7.5 Regression Testing

Regression testing ensures that new changes do not break existing features.

Examples:

- Existing ETL workflows.
- Existing reports.
- Existing database queries.
- Existing authentication.
- Existing data validation rules.

7.6 Performance Testing

Performance testing measures:

- ETL execution time.
- Query response time.
- Data loading speed.
- CPU usage.
- Memory usage.
- Concurrent query performance.

8. Code Quality Analysis

SonarQube can be integrated into the pipeline.

It checks:

- Bugs.
- Code smells.
- Duplicated code.
- Maintainability.
- Reliability.
- Security issues.
- Test coverage.

Example:

```
Code  |
v
SonarQ
ube
  |
  +---- Quality Gate PASS ----> Continue
  |
  +---- Quality Gate FAIL ----> Stop Pipeline
```

9. Security Checks

Security checks are performed before deployment.

Security activities

1. Dependency vulnerability scanning.
2. Container image scanning.
3. Secret detection.
4. Authentication testing.
5. Authorization testing.
6. Database security validation.
7. Secure configuration checking.

Trivy can scan Docker images for known vulnerabilities.

Example:

```
Docker Image
|     v Trivy
Security Scan
    |
    +---- Vulnerabilities Found ----> FAIL
    |
    +---- No Critical Issues -------> PASS
```

Passwords, API keys, and database credentials should never be stored directly in source code.

Environment variables or secret-management systems should be used.

---

10. Packaging

After successful testing and quality checks, the application is packaged into a Docker image.

Example:

```
datawarehouse:v1.0
```

The image is pushed to a container registry.

```
Build
  ↓
Docker Image
  ↓
Security Scan
  ↓
Container Registry
```

The same tested image is promoted from staging to production to avoid differences between environments.

---

11. Deployment Environments

The pipeline contains three major environments.

11.1 Development Environment

Used by developers for initial validation.

Purpose:

- Developer testing.
- Feature testing.
- Debugging.
- Initial integration testing.

Deployment can be automatic after successful CI checks.

11.2 Staging Environment

The staging environment closely represents production.

Purpose:

- Complete integration testing.
- Data-quality testing.
- Performance testing.
- User acceptance testing.
- Final verification.

Deployment occurs only after the development pipeline succeeds.

11.3 Production Environment

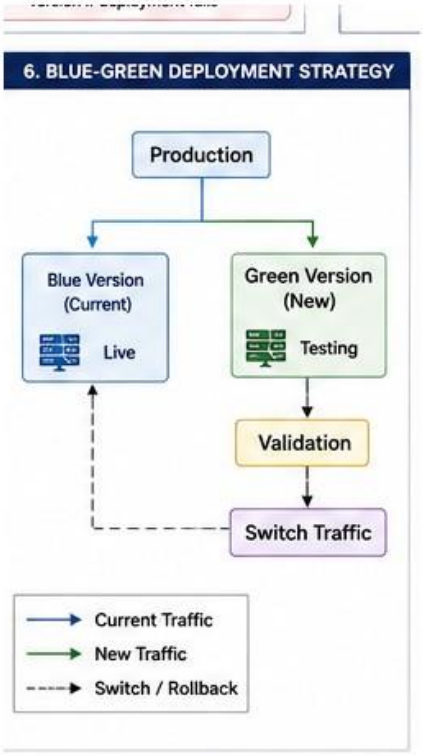
Production is the live environment used by actual users and business applications.

Production deployment requires:

- Successful automated tests.
- Successful security checks.
- Successful quality gate.
- Staging validation.
- Approval.

---

12. Deployment Strategy



A **Blue-Green Deployment** strategy can be used for production.

The current version remains available while the new version is tested.

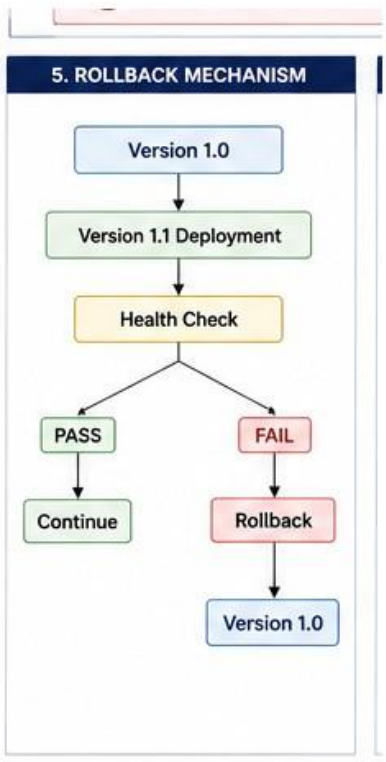
After successful validation, traffic is switched to the new version.

Advantages:

- Minimal downtime.
- Easy rollback.
- Safer production deployment.
- Quick recovery from deployment failures.

---

13. Rollback Mechanism



If the new deployment fails, the system should automatically or manually roll back to the previous stable version.

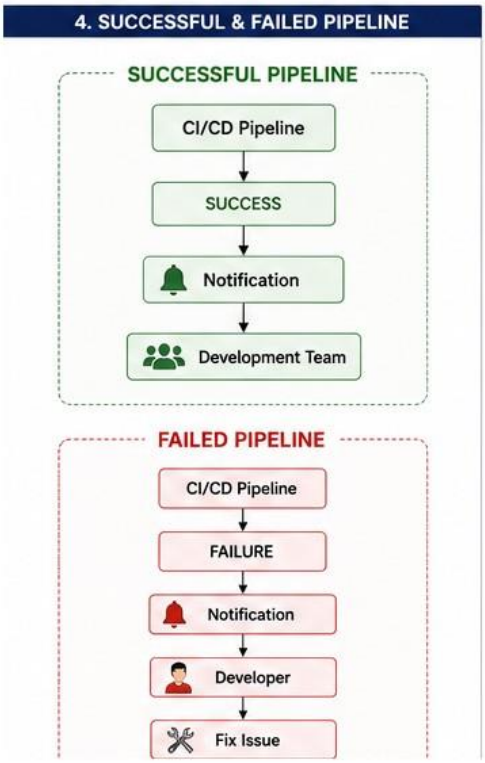
Example:

Database migrations should also be designed carefully so that incompatible changes do not prevent rollback.

Database backups should be taken before major production changes.

---

14. Notifications



The pipeline sends notifications after execution.

Successful Pipeline

Failed Pipeline

CI/CD Pipeline
↓
FAILU
RE
↓
Notification
↓
Develop
er ↓
Fix Issue

Notifications can be delivered through:

• Email.• Slack.

- Microsoft Teams.

The notification should contain:

- Build number.
- Commit ID.
- Pipeline status.
- Failed stage.
- Error information.
- Deployment status.

---

15. Monitoring

After production deployment, monitoring tools can continuously monitor the system.

Prometheus can collect metrics such as:

- CPU utilization.
- Memory usage.
- Application response time.
- Database performance.
- ETL execution time.
- Error rates.

Grafana can display these metrics using dashboards.

Monitoring helps identify production problems quickly.

---

16. Sample CI/CD Workflow

The complete workflow is:

---

17. Sample Jenkins Pipeline Configuration

A Jenkins pipeline can be represented using the following stages:

```
pipeline {
agent any

   stages {

      stage('Checkout') {          steps {              git
'https://github.com/example/datawarehouse.git'          }
}

      stage('Build') {          steps {              sh
'docker build -t datawarehouse:latest .'          }
}

      stage('Unit Tests') {
steps {              sh 'pytest
tests/unit'          }       }

      stage('Integration Tests') {
steps {              sh 'pytest
tests/integration'          }       }

      stage('Data Quality Tests') {
steps {              sh 'pytest
tests/data_quality'          }       }

      stage('Code Quality') {
steps {              sh 'sonar-
scanner'          }       }

      stage('Security Scan') {          steps {
sh 'trivy image datawarehouse:latest'          }
}

      stage('Deploy Development') {
steps {              sh 'docker compose
up -d'          }       }
```

```
        stage('Deploy Production') {            steps {
  input 'Approve Production Deployment'
  sh 'docker compose up -d'            }
        }
  }

    post {        success {            echo 'Pipeline
  completed successfully'        }

        failure {            echo
  'Pipeline failed'        }
    }
  }
```

---

18. Quality Gates

The pipeline should stop if important checks fail.

CheckExpected ResultAction		
Compilation	Successful	Continue
Unit Tests	100% required critical tests pass	Continue
Integration Tests	Pass	Continue
Data Quality	No critical errors	Continue
Code Quality	Quality gate passed	Continue
Security	No critical vulnerabilities	Continue
Staging Tests	Pass	Continue
Production Health Check	Pass	Deployment successful

If any critical quality gate fails, the production deployment is blocked.

---

19. Advantages of the Proposed CI/CD Pipeline

Faster Development

Automated builds and testing reduce manual effort.

Improved Software Quality

Automated tests detect defects before production.

Better Data Reliability

Data-quality testing prevents incorrect data from entering the warehouse.

Improved Security

Security scanning identifies vulnerabilities before deployment.

Consistent Deployment

Docker provides consistent environments across development, staging, and production.

Reduced Downtime

Blue-green deployment allows safer production releases.

Easy Rollback

The previous stable version can be restored when a deployment fails.

Continuous Monitoring

Monitoring helps identify production problems quickly.

---

20. Tool Selection Justification

Git and GitHub

Git provides version control, while GitHub provides centralized source-code management and collaboration.

Jenkins

Jenkins is suitable for automating CI/CD pipelines and supports integration with testing, Docker, SonarQube, and deployment tools.

Docker

Docker packages the application and its dependencies into a consistent container.

PyTest

PyTest provides a simple framework for automated unit, integration, and data-quality testing.

SonarQube

SonarQube automatically checks code quality and identifies bugs, code smells, and security issues.

Trivy

Trivy can scan container images and dependencies for known vulnerabilities.

PostgreSQL/MySQL

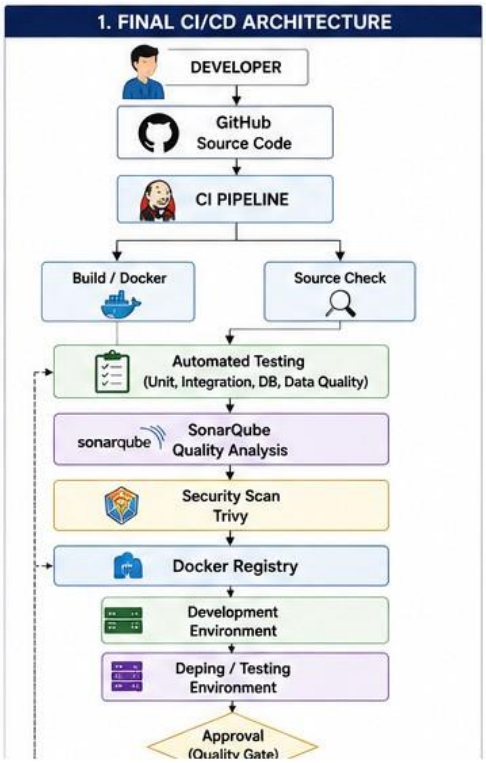
A relational database can be used for storing structured warehouse data.

Prometheus and Grafana

They provide monitoring and visualization of production metrics.

---

21. Final CI/CD Architecture



22. Conclusion

The proposed CI/CD pipeline provides an automated and reliable approach for developing and deploying the Data Warehouse Management System.

The pipeline integrates source-code management, automated building, unit testing, integration testing, database testing, data-quality testing, code-quality analysis, security scanning, containerization, staging, production deployment, monitoring, and rollback.

By automating these activities, the organization can reduce manual errors, improve software and data quality, deploy new features faster, and maintain a stable production environment.

Therefore, the proposed CI/CD architecture is suitable for a Data Warehouse Management System because it provides **automation, reliability, security, quality control, continuous delivery, and safe deployment**.